

Università di Messina — CdL in Informatica — Reti e Sistemi Distribuiti Mod.B
Hands-On #19

Algoritmi distribuiti in C: Leader Election

Last Update: May 18, 2026

Scenario

In questo HO implementeremo in C i due algoritmi di leader election studiati a lezione: **Floodmax** e **LCR**. L'architettura è identica a quella dell'HO-18: un processo per nodo, una pipe per ogni arco, sincronizzazione a quattro fasi per garantire la semantica della rete sincrona.

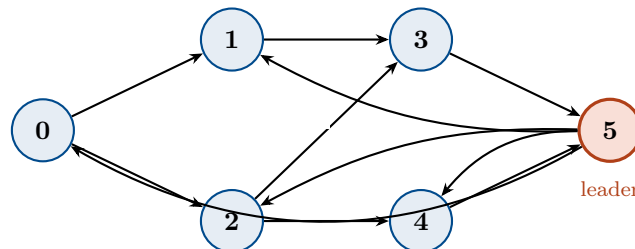
Il programma viene diviso in due eseguibili distinti: `ho19_floodmax` e `ho19_lcr`, ciascuno con la propria topologia.

File	Algoritmo	Topologia	N	Leader atteso
<code>ho19_floodmax.c</code>	Floodmax	digrafo generico	6	nodo 5
<code>ho19_lcr.c</code>	LCR	anello diretto	5	nodo 4

Le due reti

Rete Floodmax — digrafo a 6 nodi, diametro 3

Il nodo 5 ha l'UID massimo e sarà eletto leader. La costante `DIAM = 3` deve essere nota a ogni processo.

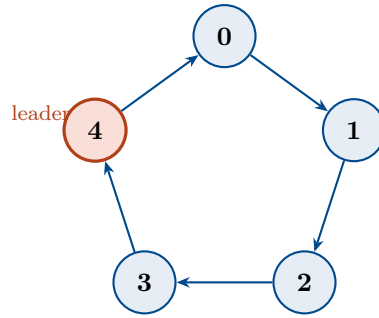


Archi in uscita da ogni nodo

0	→ 1, → 2
1	→ 3
2	→ 3, → 4
3	→ 5
4	→ 5
5	→ 0, → 1, → 2, → 4

Rete LCR — anello diretto a 5 nodi

LCR richiede una rete ad anello: ogni nodo ha esattamente un in-neighbor e un out-neighbor. La direzione è $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0$.



Strutture di stato

I due algoritmi condividono gli stessi campi `my_id`, `max_id`, `leader`, ma differiscono nel quarto campo dello stato.

Campo	Floodmax	LCR
<code>my_id</code>	UID proprio, immutabile	UID proprio, immutabile
<code>max_id</code>	massimo UID visto	massimo UID visto
<code>leader</code>	UNKNWN/IS_LDR/NOT_LDR	idem
quarto campo	<code>round</code> (contatore)	<code>snd_flag</code> (interruttore)

Il contrasto tra i due quart campi riflette la differenza fondamentale: Floodmax conta i round e termina quando `round == DIAM`; LCR non conta i round e termina quando il proprio UID torna indietro.

Il codice fornito

La struttura di pipe, fork e sincronizzazione è identica a quella dell'HO-18. Ripassate le funzioni `create_pipes()`, `close_unused_fds()`, `send_double()`, `recv_double()`, `send_ack()`, `recv_ack()` prima di iniziare.

Il loop dei round (`run_floodmax()` e `run_lcr()`) è fornito e segue le stesse quattro fasi: msg, raccolta+stf, ack, wait.

Differenza di terminazione tra i due algoritmi

Listing 1: Loop di Floodmax: $\text{ROUNDS} = \text{DIAM} + 1$

```

1  /* Floodmax: il numero di round e' fissato a DIAM + 1.
2   * Nei primi DIAM round msg() trasmette max_id.
3   * Nell'ultimo round msg() torna NULL_MSG e stf() imposta leader. */
4  for (int r = 0; r < ROUNDS; r++) { /* ROUNDS = DIAM + 1 = 4 */
5      ...
6      send_double(..., floodmax_msg(id, &s, j)); /* null se r == DIAM */
7      ...
8      floodmax_stf(id, &s, in_msgs);             /* imposta leader se r ==
          DIAM */
9      ...
10 }
```

Listing 2: Loop di LCR: $\text{ROUNDS} = N$

```

1  /* LCR: si eseguono N round.
2   * L'UID massimo percorre l'intero anello in N hop e torna al mittente.
3   * Non serve conoscere il diametro: la terminazione emerge dallo stato. */
4  for (int r = 0; r < ROUNDS; r++) { /* ROUNDS = N = 5 */
5      ...
6      send_double(..., lcr_msg(id, &s, j)); /* null se snd_flag = 0 */
  
```

```

7     ...
8     lcr_stf(id, &s, in_msgs);           /* imposta leader nel caso 2 */
9     ...
10 }

```

Task 1 — Floodmax

1.1 — Topologia

Task 1.1 — Definire la matrice di adiacenza per Floodmax

Completate `adj[N][N]` in `ho19_floodmax.c` con gli archi della rete in Sezione 2. Usate 1.0 per gli archi presenti, 0.0 per quelli assenti.

1.2 — La funzione `floodmax_msg()`

Ogni nodo trasmette `max_id` ai propri out-neighbor per i primi `DIAM` round (quando `round < DIAM`). Nell'ultimo round (`round == DIAM`) non c'è nulla da trasmettere: `max_id` è già stato propagato ovunque nei round precedenti.

Task 1.2 — Implementare `floodmax_msg()`

```

1  /* Restituisce max_id se round < DIAM, altrimenti NULL_MSG. */
2  double floodmax_msg(int id, FloodMaxState *s, int neighbor) {
3      /* DA IMPLEMENTARE */
4  }

```

1.3 — La funzione `floodmax_stf()`

La `stf` fa tre cose in sequenza:

1. Aggiorna `max_id` con il massimo ricevuto dai vicini (scorrendo `in_msgs[]` e ignorando i `NULL_MSG`).
2. Se `round == DIAM`: confronta `max_id` con `my_id` e imposta `leader` a `IS_LDR` oppure `NOT_LDR`.
3. Incrementa `round`.

Task 1.3 — Implementare `floodmax_stf()`

```

1  void floodmax_stf(int id, FloodMaxState *s, double in_msgs[N]) {
2      /* DA IMPLEMENTARE */
3  }

```

▷ Perché l'aggiornamento avviene anche nell'ultimo round

Nell'iterazione in cui `round == DIAM`, `msg()` ha restituito `NULL_MSG`, quindi `in_msgs[]` contiene solo `NULL_MSG`. L'aggiornamento di `max_id` non cambia nulla, ma il codice funziona ugualmente: il confronto con `my_id` usa il `max_id` accumulato nei round precedenti, che è corretto.

Task 2 — LCR

2.1 — Topologia

Task 2.1 — Definire la matrice di adiacenza per LCR

Completate `adj[N][N]` in `ho19_lcr.c` per l'anello $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0$. Solo `adj[i][(i+1) % N] = 1.0`; tutte le altre celle valgono 0.0.

2.2 — La funzione `lcr_msg()`

Un nodo trasmette `max_id` solo quando `snd_flag = 1`. Se `snd_flag = 0` il nodo ha già determinato che non ha nulla di utile da propagare (perché ha ricevuto un UID minore del proprio o non ha ricevuto nulla).

Task 2.2 — Implementare `lcr_msg()`

```
1  /* Restituisce max_id se snd_flag = 1, altrimenti NULL_MSG. */
2  double lcr_msg(int id, LCRState *s, int neighbor) {
3      /* DA IMPLEMENTARE */
4  }
```

2.3 — La funzione `lcr_stf()`

Sull'anello ogni nodo ha esattamente un in-neighbor, quindi `in_msgs[]` contiene al più un valore non-NULL_MSG. La `stf` identifica questo valore (`recv_id`) e applica i tre casi del modello formale:

Condizione	Azione	Significato
<code>recv_id < my_id</code> o <code>NULL_MSG</code>	<code>snd_flag = 0</code>	nulla di utile: taci
<code>recv_id == my_id</code>	<code>leader = IS_LDR, snd_flag = 0</code>	proprio UID tornato: sono il leader
<code>recv_id > my_id</code>	<code>max_id = recv_id, leader = NOT_LDR, snd_flag = 1</code>	UID maggiore: propagalo

Task 2.3 — Implementare `lcr_stf()`

```
1  void lcr_stf(int id, LCRState *s, double in_msgs[N]) {
2      /* DA IMPLEMENTARE */
3  }
```

▷ Come trovare `recv_id`

Scorrete `in_msgs[i]` per $i = 0, \dots, N-1$: il primo elemento diverso da `NULL_MSG` è il messaggio del vostro unico in-neighbor. Se non trovate nulla, ponete `recv_id = -1` (minore di qualsiasi UID valido ≥ 0) e rientrate nel Caso 1.

[!] Non aggiornare `max_id` nel Caso 1

Nel Caso 1 (`recv_id < my_id`) **non** aggiornate `max_id`. Un nodo potrebbe ricevere un UID maggiore di `max_id` corrente ma ancora minore di `my_id`: aggiornare `max_id` sarebbe sbagliato perché non migliora la conoscenza del nodo e non va propagato. Il Caso 3 è l'unico in cui `max_id` cambia.

Compilazione ed esecuzione

```
# Floodmax
gcc -Wall -Wextra -o ho19_floodmax ho19_floodmax.c && ./ho19_floodmax

# LCR
gcc -Wall -Wextra -o ho19_lcr ho19_lcr.c && ./ho19_lcr
```

Output atteso per Floodmax (ordine delle righe non deterministico):

```
=== Floodmax Algorithm (N=6, DIAM=3) ===
[FLOODMAX] nodo 0: max_id=5 leader=NO
[FLOODMAX] nodo 1: max_id=5 leader=NO
[FLOODMAX] nodo 2: max_id=5 leader=NO
[FLOODMAX] nodo 3: max_id=5 leader=NO
[FLOODMAX] nodo 4: max_id=5 leader=NO
[FLOODMAX] nodo 5: max_id=5 leader=SI (LEADER)
```

Output atteso per LCR:

```
=== LCR Algorithm (anello di 5 nodi) ===
[LCR] nodo 0: max_id=4 leader=NO
[LCR] nodo 1: max_id=4 leader=NO
[LCR] nodo 2: max_id=4 leader=NO
[LCR] nodo 3: max_id=4 leader=NO
[LCR] nodo 4: max_id=4 leader=SI (LEADER)
```

▷ Verifica rapida

In entrambi gli algoritmi, al termine tutti i nodi devono avere `max_id` uguale all'UID massimo della rete (5 per Floodmax, 4 per LCR). Se qualche nodo ha `max_id` minore, la propagazione non ha raggiunto tutti i nodi: verificate la matrice di adiacenza e la funzione `msg`.